

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Implementación de Robótica Inteligente	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil	
Módulo:	Actividad: Reto Medio Término	
Fecha: 11 de mayo del 2026		

Resumen

Para el reto de medio término se implementó primeramente un controlador de lazo cerrado y un sistema de navegación de punto a punto para el Puzzlebot utilizando ROS2, esto por medio de localización por odometría, es decir, estimación de la pose del robot a partir de los sensores, en este caso, los encoders de ambos motores. Específicamente para este caso, el controlador implementado tanto para la velocidad, como para el ángulo, es un PD, permitiendo reducir el error de manera significativa.

Igualmente, se implementó un sistema de visión artificial, esto con el fin de modificar su movimiento a partir de lo observado con la cámara integrada al robot y siguiendo lo dictado por un semáforo, siguiendo el reglamento básico de un semáforo y cómo actuar en cada uno de sus tres estados.

Teniendo en cuenta lo anterior, el comportamiento del robot Puzzlebot es cada vez más autónoma, pues además de calcular su posición gracias a las velocidades proporcionadas por los sensores, es capaz de corregir tanto su distancia al punto objetivo, como los grados de giro que necesita para llegar al siguiente punto. Además de tener la capacidad de seguir lo dictado por un semáforo, gracias a una máquina de estados que le establece quedarse en el último estado detectado por la cámara que tiene integrada.

Objetivos

El objetivo de este reto de medio término es implementar un sistema de navegación autónoma para el robot diferencial de Manchester Robotics Puzzlebot mediante ROS 2 integrando localización por odometría, navegación punto a punto, control en lazo cerrado y un sistema de visión artificial para modificar el comportamiento del robot de acuerdo con el color dictado por un semáforo. Para lograrlo, el objetivo principal de este reto es estimar en tiempo real la posición y orientación del Puzzlebot mediante odometría utilizando las velocidades angulares de las ruedas. Otro objetivo fue implementar un controlador en lazo cerrado de tipo PD para reducir tanto el error de distancia como el error angular entre la posición actual del robot y cada punto objetivo de la trayectoria. Con este controlador el robot debía orientarse hacia el punto deseado y posteriormente avanzar hasta alcanzarlo dentro de una tolerancia previamente definida.

Consecuentemente, este reto tuvo como otros objetivos el desarrollar una estrategia de navegación punto a punto, en la cual los objetivos de la posición fueran establecidos por parámetros externos. De esta manera el objetivo era que la trayectoria del robot se pudiera modificar con mayor facilidad desde un archivo de configuración. Es por ello que uno de los objetivos más importantes de este reto fue el integrar una capa de decisión basada en una máquina de estados para modificar el comportamiento del robot según el color detectado de un semáforo por un sistema de visión por computadora. Finalmente, en este reto se buscó analizar el desempeño del sistema mediante métricas objetivas como el error de la posición final, el error de distancia, el error angular y la respuesta del comando de velocidad `/cmd_vel`.

Introducción

A lo largo de la ejecución de este reto se ha podido entender que la robótica integra distintas áreas de la ingeniería como la localización, el control automático, la visión artificial y la comunicación entre sistemas. En este reto se trabajó con el Puzzlebot con el fin de hacer que el robot se desplazara de manera autónoma a través de una trayectoria punto a punto y modificar su comportamiento de acuerdo al color detectado de un semáforo. Es por ello que este sistema no solo debía calcular su posición y corregir su movimiento sino también incorporar un algoritmo basado en visión artificial.

Primeramente, resulta importante definir que es un robot diferencial. Un robot diferencial es un robot que se desplaza mediante dos ruedas motrices colocadas sobre un mismo eje y donde cada rueda puede girar de manera independiente. Este tipo de configuración permite que el movimiento lineal y angular del robot dependa de las dos

ruedas. Es decir, si ambas ruedas giran a la misma velocidad entonces el robot avanza en línea recta; si una rueda gira más rápido que otra entonces el robot cambia su orientación y si giran en sentidos contrarios entonces el robot puede rotar sobre su propio eje [1].

Es por ello que para que el robot pudiera llegar a la posición deseada se implementó un sistema de control en lazo cerrado, en el cual la posición y orientación estimadas del Puzzlebot se utilizan como retroalimentación para calcular el error respecto a una referencia. A partir de ese error el controlador genera una acción que corrige y modifica el movimiento del robot. En este reto se utilizó una estructura PD para el control de la distancia y orientación. Primeramente, el término proporcional permite generar una respuesta de control directamente relacionada con el tamaño del error, mientras que el término derivativo ayuda a reducir cambios abruptos y a mejorar la estabilidad de la respuesta. En este caso, no se utilizó un integral, ya que el término integral puede acumular error cuando las señales de control se saturan o cuando existen retardos, lo que puede generar oscilaciones o comportamientos no deseados. Por ende, en el caso de esta solución del reto el controlador PD representó una alternativa adecuada para una navegación punto a punto simple [2].

Consecuentemente, de acuerdo con Ortiz del Castillo, la robustez de un controlador se refiere a la capacidad de un sistema de control de mantener un comportamiento estable y un desempeño aceptable aun y cuando existen perturbaciones, incertidumbres en el modelo, ruido o retardos de comunicación [3]. En el caso del Puzzlebot, estas condiciones pueden presentarse por el deslizamiento o desgaste de las ruedas, errores en la odometría, latencia entre nodos de ROS o saturación en los comandos de velocidad. Por ende, un controlador robusto no solo debe reducir el error en orientación y posición sino también debe ser capaz de evitar respuestas bruscas u oscilaciones. En el caso de este reto, la robustez del controlador se implementa mediante el uso de límites de velocidad, tolerancias de llegada, separación entre giro y avance y un control PD que permite responder al error actual.

Tomando en cuenta lo anteriormente expuesto, además del control de movimiento se incorporó procesamiento de imágenes en una tarjeta embebida Jetson la cual permitió capturar la imagen del entorno y entonces el algoritmo de visión procesó localmente la información visual para identificar el color del semáforo. Hablando un poco a profundidad de la tarjeta embebida, la interconexión entre la Jetson y la cámara permite que el sistema de visión artificial reciba imágenes del entorno en tiempo real. Dependiendo del tipo de cámara utilizada, la conexión puede realizarse mediante interfaces como USB, Ethernet o MIPI CSI-2, las cuales son compatibles con la Jetson [4].

Asimismo, la visión por computadora, la cual es usada en este reto, permite que los sistemas robóticos interpreten información visual captada por una cámara para tomar decisiones sobre su entorno. En aplicaciones de robotica una imagen suele pasar por etapas como la conversión del espacio de color, segmentación, detección de contornos y análisis de formas. Estas aplicaciones pueden ser realizadas a través de la librería OpenCV, la cual permite hacer operaciones de visión por computadora. Por ende, la robustez en procesamiento de imágenes se relaciona con la capacidad del algoritmo para mantener detecciones confiables ante cambios de iluminación, sombras, movimiento, ruido de cámara o pérdidas del objeto. Para mejorar la robustez de las imágenes suelen utilizarse filtros, umbrales u operaciones morfológicas como erosión, dilatación, apertura y cierre, que ayudan a limpiar la imagen y mejorar su segmentación. Finalmente, podemos decir que en este reto se integró la localización, el control en lazo cerrado, navegación punto a punto y visión artificial en una arquitectura distribuida basada en ROS 2.

Solución del problema

La solución se diseñó como un sistema distribuido en ROS 2, organizado en cuatro nodos independientes que se comunican exclusivamente a través de tópicos. Esta separación permite probar y reemplazar cada bloque por separado, y refleja una arquitectura limpia donde cada nodo tiene una sola responsabilidad: estimar la pose, planear la trayectoria, controlar el movimiento, o percibir el entorno. Los cuatro nodos del paquete `'puzzlebot_tuned_controller'` son:

- `puzzlebot_odometry`: estima la pose (x, y, θ) del robot integrando las velocidades de las ruedas.
- `path_generator_node`: gestiona una lista de waypoints y publica el objetivo activo.
- `controller_path`: controlador PID dual (distancia y ángulo) que genera comandos de velocidad para alcanzar el objetivo.
- `traffic_light_node`: nodo de visión que clasifica el color del semáforo a partir de la cámara CSI de la Jetson.

El flujo de datos es el siguiente. La odometría escucha los tópicos de los encoders (`/VelocityEncR`, `/VelocityEncL`), publica continuamente la pose estimada en `/odom` y emite la transformación TF `odom → base_link`. El generador de trayectoria también consume `/odom` para saber cuándo ha llegado a un waypoint y, en ese caso, publica el siguiente objetivo en `/goal`. El controlador PID se suscribe simultáneamente a `/odom`, `/goal` y `/detected_color`; con

esa información calcula el comando de velocidad lineal y angular que envía al robot por `/cmd_vel`. Finalmente, el nodo de visión publica el color detectado en `/detected_color`, lo que permite al controlador modular su salida según el estado del semáforo.

Toda la configuración (radios de rueda, ganancias del PID, lista de waypoints, tolerancias) se externalizó al archivo `config/puzzlebot_params.yaml` y se carga vía los launch files (`square_controller.launch.py` y `path_controller.launch.py`). Esto permite ajustar el comportamiento sin recompilar el paquete.

Nodo de odometría: `puzzlebot_odometry`

La localización del robot se resuelve mediante dead reckoning, es decir, integración numérica de las velocidades angulares de las ruedas para obtener la pose en el plano. El nodo se suscribe a los tópicos `/VelocityEncR` y `/VelocityEncL`, donde el firmware del Puzzlebot publica las velocidades en rad/s de cada rueda. Estas suscripciones se configuran con un perfil de calidad de servicio `BEST_EFFORT`, ya que la fuente del Puzzlebot publica con ese perfil; un perfil `RELIABLE` provocaría que los mensajes nunca lleguen.

Sobre las velocidades recibidas se aplica el modelo cinemático del robot diferencial:

$$v = r \cdot (\omega_R + \omega_L) / 2$$

$$\omega = r \cdot (\omega_R - \omega_L) / L$$

donde $r = 0.05$ m es el radio de las ruedas y $L = 0.19$ m la separación entre ellas. La pose se actualiza por el método de Euler con un período de muestreo $T_s = 0.02$ s (50 Hz):

$$x(k+1) = x(k) + v \cdot \cos(\theta) \cdot T_s$$

$$y(k+1) = y(k) + v \cdot \sin(\theta) \cdot T_s$$

$$\theta(k+1) = \theta(k) + \omega \cdot T_s$$

El ángulo θ se normaliza al intervalo $(-\pi, \pi]$ mediante `atan2(sin( $\theta$ ), cos( $\theta$ ))` para evitar que se acumulen vueltas que no representan información útil. Antes de publicar, el yaw se convierte a cuaternión, que es el formato exigido por el mensaje `nav_msgs/Odometry`. Además del mensaje en `/odom`, el nodo emite la transformación `odom` $\rightarrow$ `base_link` mediante `TransformBroadcaster`.

Es importante señalar que esta odometría no realiza ningún tipo de fusión sensorial ni corrección por deslizamiento; el error es acumulativo. Sin embargo, para trayectorias cortas y velocidades moderadas como las que requiere este reto, la precisión es suficiente para cerrar el lazo de control.

Generador de trayectoria: path_generator_node

La responsabilidad de este nodo es mantener una lista de waypoints definida por parámetros (points_x, points_y en el YAML) y, en cada momento, publicar como objetivo activo el waypoint que el robot debe perseguir. La lista se ejecuta secuencialmente: cada iteración del timer (10 Hz) consulta la pose más reciente del robot, calcula la distancia euclidiana al waypoint actual y, si es menor a arrival_tolerance (por defecto 0.15 m), incrementa el índice y pasa al siguiente. Cuando se agota la lista, el nodo marca la misión como completada y deja de publicar.

El mensaje publicado en /goal es de tipo MakbetsPose (definido en el paquete makbets_pose), que además de las coordenadas (x, y) incluye los límites de velocidad lineal y angular.

Controlador PID: controller_path

El controlador es el núcleo del lazo cerrado. Implementa dos PIDs independientes, uno para el error de distancia al objetivo y otro para el error angular, cuyas salidas se mapean a las componentes lineal y angular del comando Twist publicado en /cmd_vel.

Cálculo de errores

En cada iteración del timer ($dt = 0.05$ s, 20 Hz), a partir de la pose actual (x_R , y_R , θ_R) y del objetivo (x_G , y_G), se calculan:

$$dx = x_G - x_R$$

$$dy = y_G - y_R$$

$$\text{dist_error} = \sqrt{dx^2 + dy^2}$$

$$\text{desired_}\theta = \text{atan2}(dy, dx)$$

$$\text{theta_error} = \text{normalize_angle}(\text{desired_}\theta - \theta_R)$$

La normalización de theta_error asegura que el robot siempre gire por el camino más corto.

Estructura PID

Cada PID se evalúa con la ecuación clásica en forma discreta:

$$u(k) = K_p \cdot e(k) + K_i \cdot \sum e \cdot dt + K_d \cdot (e(k) - e(k-1))/dt$$

Las ganancias se exponen como parámetros declarados (kp_dist, ki_dist, kd_dist, kp_theta, ki_theta, kd_theta) y se cargan desde el YAML. La sintonización final que se utilizó es:

Parámetro	Distancia	Ángulo
Kp	0.99	0.7
Ki	0.0	0.0
Kd	0.06	0.01

El término integral en distancia y angular se mantuvo en cero porque el sistema se comporta de forma estable sin él. La derivada en el ángulo se subió respecto a la versión inicial para amortiguar el sobreimpulso angular durante los giros.

Estrategia de movimiento "girar primero, luego avanzar"

El controlador adopta una política sencilla pero robusta: si el error angular supera la tolerancia ($\theta_{\text{tolerance}} = 0.1 \text{ rad} \approx 5.7^\circ$), el robot solo gira (velocidad lineal forzada a cero) hasta alinearse con el objetivo. Una vez dentro de la tolerancia, comienza a avanzar mientras sigue corrigiendo finamente la orientación. Esta estrategia evita trayectorias en arco amplias al inicio del segmento y produce un movimiento más predecible para waypoints separados, a costa de un movimiento ligeramente menos suave en las transiciones.

Anti-windup en el integrador angular

Aunque actualmente $k_i_{\theta} = 0$, se dejó implementado un esquema de anti-windup por clamping: el integral solo se acumula cuando $|\theta_{\text{error}}| < 0.5$ rad. Esto previene que, ante errores grandes (por ejemplo, al recibir un waypoint detrás del robot), el integrador acumule un valor enorme que tarde en disiparse y produzca sobreimpulso al cruzar el setpoint.

Reset de estado al recibir un nuevo objetivo

Una de las correcciones importantes durante el desarrollo fue el manejo de transiciones entre waypoints. Cuando llega un nuevo objetivo en `/goal`, el callback compara las nuevas coordenadas con las anteriores usando `math.isclose(abs_tol=1e-3)` (1 mm de tolerancia, en vez de comparación directa con `'=='`, que es frágil con flotantes). Si el objetivo cambió, se resetean los integrales y los errores previos de ambos PIDs:

```
self.integral_dist = 0.0
```

```
self.integral_theta = 0.0
```

```
self.prev_dist_error = 0.0
```

```
self.prev_theta_error = 0.0
```

Sin este reset, los términos derivativo e integral acumulados del waypoint anterior contaminan la respuesta inicial al nuevo objetivo, provocando comandos espurios.

Máquina de estados con el semáforo

El controlador es además una máquina de estados con tres modos derivados del color publicado en `/detected_color`:

- STOP (rojo): factor de velocidad 0.0. El controlador publica explícitamente `cmd = (0, 0)` y retorna inmediatamente del callback. Esto es crítico: en una versión previa del código, el `return` faltaba y el resto del lazo se seguía ejecutando, lo que provocaba que el integrador siguiera acumulando con el robot detenido y soltara un "comandazo" al pasar a verde. La corrección consistió en añadir el `return` después de publicar el cero.
- SLOW (amarillo): factor 0.3. Los límites `v_max` y `w_max` recibidos en el mensaje `MakbetsPose` se multiplican por este factor, reduciendo la velocidad de manera proporcional sin alterar la dinámica del PID.
- GO (verde): factor 1.0, operación normal.

El estado solo cambia cuando el color recibido es distinto al actual, lo cual reduce el ruido en los logs y evita transiciones espurias por mensajes “NONE” (ausencia de detección).

Saturación de salidas

Antes de publicar, el comando se satura a los límites recibidos del planeador para garantizar que el robot no excede sus capacidades físicas:

$$\text{cmd.linear.x} = \max(0.0, \min(v_max, v_out))$$
$$\text{cmd.angular.z} = \max(-w_max, \min(w_max, w_out))$$

Logging estructurado para depuración

El nodo incluye un sistema de logging diseñado específicamente para depuración en campo. Cada 0.5 segundos (10 ciclos a 20 Hz) imprime un status periódico con la pose actual, el goal, los errores y los comandos generados; adicionalmente, se loguea cada transición de modo ('GIRO $\leftrightarrow$ AVANCE') y cada cambio de estado del semáforo, lo que permite reconstruir el comportamiento del robot posteriormente sin saturar la terminal.

Detección del semáforo: traffic_light_node

La percepción del semáforo se implementó como un nodo independiente que corre directamente en la Jetson, accediendo a la cámara CSI a través de un pipeline GStreamer (nvarguscamerasrc). El uso de GStreamer es necesario porque la Jetson no expone la cámara CSI como un dispositivo /dev/video* estándar; el pipeline configura la captura a 1280×720 a 30 fps, aplica nvvidconv para volcar el frame de la memoria NVMM a memoria de CPU, y entrega el resultado a OpenCV en formato BGR.

Región de interés

Para reducir falsos positivos provenientes del entorno, el procesamiento se restringe a una ROI central del frame, que ocupa el 20% del ancho y el 60% del alto. Esto asume que el robot está alineado con el semáforo cuando se aproxima, y descarta colores en los bordes del campo de visión (cinta, marcas, otros objetos).

Espacio de color HSV y umbralización

El frame se desenfoca con un kernel Gaussiano de 7×7 para suavizar ruido de captura y se convierte a HSV, sobre la imagen HSV se aplican rangos para cada color objetivo:

- Verde: un solo rango $H \in [40, 85]$.
- Amarillo: un solo rango $H \in [16, 40]$, con saturación y valor moderados.
- Rojo: dos rangos ($H \in [0, 10]$ y $H \in [170, 179]$) que se unen con bitwise_or. Esto es necesario porque el rojo se "envuelve" en el extremo del círculo de matiz HSV.

A las máscaras se les aplican operaciones morfológicas: MORPH_OPEN para eliminar puntos aislados (ruido) y, en el caso del amarillo, también MORPH_CLOSE para cerrar pequeños huecos.

Filtros de forma para el amarillo

Durante el desarrollo se observó que el amarillo es el color más propenso a falsos positivos, ya que abundan superficies amarillentas en el entorno. Por eso se le añadieron filtros geométricos sobre los contornos detectados: se descartan contornos con área fuera del rango $[200, 2000]$, con circularidad < 0.6 . Solo si un contorno cumple los criterios se considera un foco de semáforo válido.

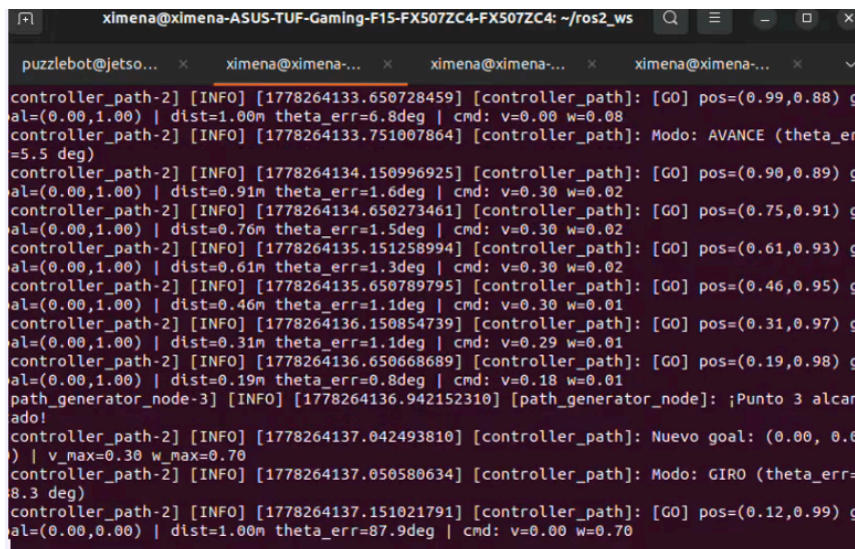
Umbral de área para validación de distancia

Una decisión clave fue calibrar umbrales mínimos de área por color (AREA_UMBRAL_VERDE = 1650, AREA_UMBRAL_AMARILLO = 1180, AREA_UMBRAL_ROJO = 1380 píxeles). Estos valores se obtuvieron experimentalmente colocando el semáforo a aproximadamente 25 cm de la cámara y midiendo el área de la máscara correspondiente. Si el área observada en tiempo de ejecución no supera el umbral, el color se ignora aunque sea el dominante. El efecto práctico es que el robot solo reacciona al semáforo cuando lo tiene cerca, evitando que detecciones lejanas o parciales alteren su comportamiento.

Los términos integrales se mantuvieron en cero. La razón es que, en la práctica, la odometría tiene suficiente precisión para que el error estacionario sea despreciable, y activar los integrales sin un anti-windup robusto introduce más problemas (sobreimpulso, oscilaciones) que beneficios. El esqueleto del integral angular se dejó implementado con su clamp por si en una iteración futura se necesita corregir un sesgo sistemático.

Resultados

Los resultados obtenidos fueron consistentes con los resultados previamente esperados. El robot recorre un camino construido previamente con puntos definidos como parámetros para el controlador. El camino generado por el robot se encuentra dentro del margen de ± 10 cm con respecto al camino teórico. Para esta fase del proyecto se puso a prueba el funcionamiento de los nodos creados en ROS 2 con dos tipos de caminos: cuadrado y triángulo. Por simplicidad, se realizó el experimento teniendo la base y altura de ambas figuras iguales a 1 metro.



```
ximena@ximena-ASUS-TUF-Gaming-F15-FX507ZC4-FX507ZC4: ~/ros2_ws
puzzlebot@jetso... ximena@ximena... ximena@ximena... ximena@ximena...
controller_path-2] [INFO] [1778264133.650728459] [controller_path]: [GO] pos=(0.99,0.88) g
al=(0.00,1.00) | dist=1.00m theta_err=6.8deg | cmd: v=0.00 w=0.08
controller_path-2] [INFO] [1778264133.751007864] [controller_path]: Modo: AVANCE (theta_er
=5.5 deg)
controller_path-2] [INFO] [1778264134.150996925] [controller_path]: [GO] pos=(0.90,0.89) g
al=(0.00,1.00) | dist=0.91m theta_err=1.6deg | cmd: v=0.30 w=0.02
controller_path-2] [INFO] [1778264134.650273461] [controller_path]: [GO] pos=(0.75,0.91) g
al=(0.00,1.00) | dist=0.76m theta_err=1.5deg | cmd: v=0.30 w=0.02
controller_path-2] [INFO] [1778264135.151258994] [controller_path]: [GO] pos=(0.61,0.93) g
al=(0.00,1.00) | dist=0.61m theta_err=1.3deg | cmd: v=0.30 w=0.02
controller_path-2] [INFO] [1778264135.650789795] [controller_path]: [GO] pos=(0.46,0.95) g
al=(0.00,1.00) | dist=0.46m theta_err=1.1deg | cmd: v=0.30 w=0.01
controller_path-2] [INFO] [1778264136.150854739] [controller_path]: [GO] pos=(0.31,0.97) g
al=(0.00,1.00) | dist=0.31m theta_err=1.1deg | cmd: v=0.29 w=0.01
controller_path-2] [INFO] [1778264136.650668689] [controller_path]: [GO] pos=(0.19,0.98) g
al=(0.00,1.00) | dist=0.19m theta_err=0.8deg | cmd: v=0.18 w=0.01
path_generator_node-3] [INFO] [1778264136.942152310] [path_generator_node]: ¡Punto 3 alcan
ado!
controller_path-2] [INFO] [1778264137.042493810] [controller_path]: Nuevo goal: (0.00, 0.0
) | v_max=0.30 w_max=0.70
controller_path-2] [INFO] [1778264137.050580634] [controller_path]: Modo: GIRO (theta_err=
8.3 deg)
controller_path-2] [INFO] [1778264137.151021791] [controller_path]: [GO] pos=(0.12,0.99) g
al=(0.00,0.00) | dist=1.00m theta_err=87.9deg | cmd: v=0.00 w=0.70
```

Imagen 1. Representación del avance del robot en terminal.

Como se puede ver en la *Imagen 1*, el nodo del controlador muestra en pantalla una actualización del avance del robot, indicando si está en su etapa de movimiento lineal o angular, si ha alcanzado cierto punto del camino predefinido, o si ha finalizado su movimiento.

Asimismo, se utilizó la información del tópico `/detected_color` para mostrar los colores detectados por la cámara, los cuales modificarían el avance del robot. Una vez que el robot detecta el color amarillo, comienza a avanzar más lentamente, para posteriormente detenerse una vez que el semáforo cambia a color rojo. En este punto, se pone en pausa la funcionalidad del controlador para evitar una acumulación excesiva en el error en la posición del robot (lo cual perjudicaría el resto de la ejecución del robot). De igual manera, el robot y el controlador no reanudarán la tarea hasta que el semáforo cambie a verde y el color sea detectado por la cámara. En caso de que la cámara no detecte *verde*, *amarillo* o *rojo*, el

mensaje en el t pico ser  NONE, y la m quina de estados permanece en el  ltimo estado gobernado por estos 3 colores.

El siguiente video expone una demostraci n visual del funcionamiento de esta implementaci n: <https://youtu.be/kJFoRRCo8pw?feature=shared>

Integraci n y flujo completo

El comportamiento integrado del sistema durante una ejecuci n t pica es el siguiente. Al lanzar el sistema, los cuatro nodos se inician en paralelo. La odometr a empieza a publicar la pose en (0, 0, 0) y se actualiza con cada lectura de los encoders. El generador de trayectoria publica el primer waypoint de la lista en /goal. El controlador, al recibir el goal, calcula los errores y comienza a girar hacia  l si la diferencia angular supera la tolerancia. En paralelo, el nodo de visi n est  observando el sem foro: mientras detecta verde, el factor de velocidad se mantiene en 1.0 y el robot avanza con normalidad. Si la c mara observa un foco amarillo v lido y suficientemente cercano, el controlador escala las velocidades por 0.3. Si detecta rojo, el controlador publica cmd = (0, 0) y el robot se detiene completamente, sin acumular estado interno (gracias al return temprano), hasta que vuelva a observar verde. Cuando el robot llega al waypoint dentro de la tolerancia (goal_tolerance = 0.1 m), el generador de trayectoria avanza al siguiente y el ciclo se repite hasta agotar la lista, momento en el que el sistema queda en reposo.

Conclusiones

La arquitectura distribuida en cuatro nodos independientes (puzzlebot_odometry, path_generator_node, controller_path y traffic_light_node) demostr  ser una decisi n acertada, ya que permiti  desarrollar, probar y depurar cada bloque por separado, adem s de facilitar la incorporaci n del sistema de visi n sin modificar la l gica de control ya validada. En cuanto al desempe o del controlador PD, los resultados experimentales confirmaron que el robot es capaz de seguir las trayectorias cuadrada y triangular (de 1 metro por lado) manteni ndose dentro del margen de error permitido de ± 10 cm respecto a la trayectoria te rica. La estrategia de "girar primero, luego avanzar" result  adecuada para waypoints separados, y el reset de los t rminos previos del PID al recibir un nuevo objetivo evit  comportamientos espurios en las transiciones entre puntos. Sin embargo, es importante mencionar una limitaci n observada durante las pruebas: el controlador no produce

exactamente el mismo resultado en ejecuciones repetidas. La razón es que el cálculo del término derivativo asume un Δt constante de 0.05 s (correspondiente a la frecuencia de 20 Hz del timer), pero en la práctica este intervalo presenta variaciones debido a la latencia en la comunicación entre nodos, la carga del sistema operativo y los tiempos de procesamiento en la Jetson. Estas variaciones se traducen en pequeñas diferencias en la respuesta del controlador entre una ejecución y otra. A pesar de esto, el sistema demostró ser lo suficientemente robusto como para que el error final se mantenga consistentemente dentro de la tolerancia de ± 10 cm, lo cual es aceptable para los objetivos del reto.

Por el lado del sistema de visión, la combinación de procesamiento en espacio HSV, operaciones morfológicas, filtros geométricos de circularidad y aspect ratio para el amarillo, y umbrales de área calibrados a una distancia conocida (~ 25 cm) permitió obtener una detección confiable del semáforo en tiempo real sobre la Jetson. La integración con el controlador a través de una máquina de estados de tres modos (STOP, SLOW, GO) hizo que el robot responda correctamente al semáforo, deteniéndose en rojo sin acumular error interno gracias al return temprano en el lazo de control.

Finalmente, como trabajo futuro, valdría la pena medir explícitamente el Δt real entre iteraciones del controlador y usarlo en el cálculo del derivativo, así como considerar la fusión sensorial con una IMU para reducir el error acumulativo de la odometría en trayectorias más largas.

Referencias

1. Robóticos. (s.f.). Introducción al Robot Móvil Diferencial.
<https://es.scribd.com/document/894377792/Manual-robot-movil-diferencial>
2. Universidad Tecnológica de Tamaulipas Norte. (2025). Características del Controlador PD en Sistemas de Control.
<https://www.studocu.com/es-mx/document/universidad-tecnologica-de-tamaulipas-norte/sistemas-mecanicos/caracteristicas-del-controlador-pd-en-sistemas-de-control/128099866>
3. Ortiz del Castillo. (2005). ROBUSTEZ EN CONTROLADORES RETROALIMENTADOS.
<http://www.iqcelaya.itc.mx/~richart/TesisDoctorado/2005%20Ort%C3%ADz%20del%20Castillo.pdf>
4. NVIDIA Developer. (s.f.). Taking Your First Picture with CSI or USB Camera.
https://developer.nvidia.com/embedded/learn/tutorials/first-picture-csi-usb-camera?utm_source=chatgpt.com